

JavaScript: The New Parts

This is the fifth part of our series on ECMAScript 2015. Read on to understand more about default parameters, rest parameters and spread operator.

JavaScript language is evolving more rapidly than in the past. New features are getting added every year. So let's learn more about the latest features in order to make your programming easier, bug-free and maintainable.

One of the unique features of JavaScript is the support for functions in the language. Functions are first class citizens, which means you can do all the operations using functions that you can do with other primitive data types. ECMAScript 2015 introduces features that make functions more powerful and convenient. In this issue, we will learn about features related to function parameters, namely, default parameters, rest parameters and spread operators.

Default parameters

A function can have zero or more parameters. If a function has two parameters, it is expected that two values are passed when calling the function. If the number of parameters passed is less, the JavaScript interpreter assumes the value of the missing parameters as undefined. There is no exception thrown. This implicit assignment of undefined parameters can lead to issues. Thankfully, IDEs show function signatures for programmers, which helps to avoid parameters being missed. But as a good programming practice, it is required that you handle these scenarios as part of the function.

```
function fn(one, two, three) {
    console.log(one, two, three);
}
fn(1);
```

Output:
1, undefined, undefined

In ES6, we have a new feature called default parameters, which serves the purpose of giving a value to a parameter other than *undefined*. At the time of calling the function, it need not include a value for these parameters; these are optional. This also avoids conditional statements to check *undefined*, all through your function. This feature is there in other languages like Python, C++ and C#. It is also called default *arguments* (Wikipedia).

Shown below is a function for computing simple interest which takes the principal, time and rate of interest. Let's assume that in every call the principal has a value, and the time and rate of interest are fixed at 1 and 8, respectively, when no value is passed.

```
1. function simple_interest(p, t, r) {
2.   let time = t || 1;
3.   let rate = r || 8;
```

```
4.   let si = (p * time * rate)/100;
5.   console.log("p =" + p + ", t =" + time +
6.     " , r =" + rate + " ; si =" + si);
7. }
8. simple_interest(1000, 2, 10);
9. simple_interest(1000, 5);
10. simple_interest(1000);
11. simple_interest(1000, 2, undefined);
12. simple_interest(1000, 2, null);
13. simple_interest(1000, 2, 0);
```

The output is:

```
p =1000, t = 2, r = 10; si = 200
p =1000, t = 5, r = 8; si = 400
p =1000, t = 1, r = 8; si = 80
p =1000, t = 2, r = 8; si = 160
p =1000, t = 2, r = 8; si = 160
p =1000, t = 2, r = 8; si = 160
```

Check this example at <http://www.es6fiddle.net/ii88u9ol/>.

In the above example, in Lines 11, 12 and 13 we are passing *undefined*, null and 0. For all these three cases, Line 3 evaluates the OR condition and assigns the default rate of interest to 8.

Let us write the same function using default parameters as shown below. Along with function parameters, we are also giving default values to the time and rate of interest. Again, here the assumption is that at least the principal amount is passed at the time of calling.

```
1. function simple_interest(p, t=1, r=8) {
2.   let si = (p * t * r)/100;
3.   console.log("p =${p}, t =${t},
4.     r =${r}; si =${si}");
5. }
6. simple_interest(1000, 2, 10);
7. simple_interest(1000, 5);
8. simple_interest(1000);
9. simple_interest(1000, 2, undefined);
10. simple_interest(1000, 2, null);
```

The output is:

```
p = 1000, t = 2, r = 10; si = 200
p = 1000, t = 5, r = 8; si = 400
p = 1000, t = 1, r = 8; si = 80
p = 1000, t = 2, r = 8; si = 160
p = 1000, t = 2, r = null; si = 0
```